

Revisions R0F1A

1 - Feature detection & description

ImageNet: 1000 classes, millions of pictures used to train models like ResNet and VGG

models invariant to rotation on ImageNet?

YES! Flipped/rotated objects like cars remain the same

→ changing angles or flipped image shouldn't be an issue for classifiers on ImageNet

For MNIST, same question but for:

1) small rotations ($\approx 20^\circ$)?

hand written digits → trickier and more sensitive to rotations

small changes, should be fine

digits remain differentiable under those conditions

NO PROBLEM if angle not too big!

2) big rotations (90°)?

Big problem! ex: 2 digits applied 90° rotation would look the same, like a 6 and 9 (in 2 different directions)

this rotation does not help generalize, it actually mixes the classes → impossible to learn

Bag of Words:

1) local feature extraction

→ extract local descriptors using SIFT for example
descriptors captures local textures, partially invariant to rotation
an image becomes many small patches with extracted features

2) visual vocabulary

vectors of descriptors are continuous
→ we need to discretize them

K-Means:

- collect many descriptors with many images
- define a vocabulary size K
- run k-means with K clusters on the descriptors
- each cluster is now a "word"

we use this algo to group descriptors together

$$\text{Loss: } C(w) = \sum_{i=1}^n \min_k \|x_i - w_k\|^2$$

← minimize distance of each point to a cluster centroid (its own cluster)

assign points to nearest centroid
move centroid to the mean of its assigned points } repeat

limits: memory intensive algo, very sensitive to the init of clusters and of dictionary size

2. Support Vector Machines

supervised learning model, learns a decision boundary with a maximum margin between classes

core idea:

we have labeled data (x_i, y_i) where $y_i \in \{-1, 1\}$

we want $f(x_i) = w^T x_i + b$

such that:

$$y_i (w^T x_i + b) \geq 1$$

to maximize the margin between classes, this means that we minimize $\|w\|$

This gives us the **hard-margin** optimization problem:

$$\min_{w, b} \frac{1}{2} \|w^*\|^2 \quad \text{s.t.} \quad y_i (w^T x_i + b) \geq 1$$

However, data not always perfectly separable. So we want to allow some errors but under constraints

so we define the **soft-margin** optimization problem:

$$\min_{w, b} \frac{1}{2} \|w^*\|^2 + C \sum_{i=1}^N \epsilon_i$$

where $\forall_i : \epsilon_i \geq 0$ and $y_i f(x_i) \geq 1 - \epsilon_i$

The **slack variables** allows us to violate the margin constraint but at a cost

if $\epsilon_i = 0$, correct classification

if $0 < \epsilon_i < 1$, correct classification but inside the margin

if $\epsilon_i = 1$, on the decision boundary (doesn't know how to classify)

if $\epsilon_i > 1$, incorrect classification

In the optim problem, we want to minimize both the margin $\|w\|^2$ but also minimize the sum $\sum \epsilon_i$

There is also the hyperparameter C .

If C is set to a large value, then violations are costly. Since penalty is expensive, it pushes the optim problem to have $\epsilon_i = 0$

This means:

- smaller margin (less tolerance)
- forces more accurate classification
- sensitive to outliers
- ϵ_i forced to reach 0

overall, low bias but high variance

But if C is set to a smaller value:

- violations are cheaper
- margin can be wider
- more robust boundary
- ϵ_i can be positive, allow errors (for outliers for example)

so we have a high bias but low variance

SVMs use the Hinge loss:

$\sum_i \max(0, 1 - y_i f(x_i))$ and if we go back to the definition of ϵ_i , we see that

$$\epsilon_i = \max(0, 1 - y_i f(x_i))$$

so the loss of the SVM is exactly the sum of the slack variables.

Key idea:

C is the trade-off between maximizing the margin for stricter classification, and minimizing the violations of the margins, to allow some errors in the classification.

Kernel trick:

data not always linearly separable in current space.

But if we increase dimensions then it can become linearly separable
assume transformation $\phi(x)$ increases dimension

we don't know the size d such that $\phi(x) \in \mathbb{R}^d$
could be huge or infinite!

Kernel trick allows us to bypass the actual transformation

We define a kernel function:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

it corresponds to the inner product in some feature space

the kernel can be different operations but skip for now

K defines the similarity of 2 points x and x' in a different space

Interpretation of values of kernel function:

$$k(x, x') = \begin{cases} ++ \text{ value} \rightarrow \text{very similar} \\ 0 \rightarrow \text{orthogonal vectors so unrelated} \\ -- \text{ value} \rightarrow \text{dissimilar} \end{cases}$$

3. Deep Vision Architectures

CNN:

used mostly for images

applies filters to pixel patches to extract local information, such as edges, corners, textures etc...

traditionally, images get reduced in size in the network while keeping relevant information

→ reduce dimensions but keep info

CNN vs. MLP (for images)

an MLP takes as input a flat vector (flattened images)

it considers all inputs, here pixels, to be independent from one another

but it's not the case!

groups of pixels give info, not individual pixels

so for small simple images (like MNIST) it's fine

but for complex large images, doesn't work

Recap of famous models

AlexNet (2012):

5 conv layers with relu and maxpooling (+LRN first 2 layers)

3 FC layers at the end with relu and dropout

Final layer outputs 1000 neurons → softmax

relu → faster training

dropout → reduce overfitting

input: $224 \times 224 \times 3$

62 million params

local response
normalization

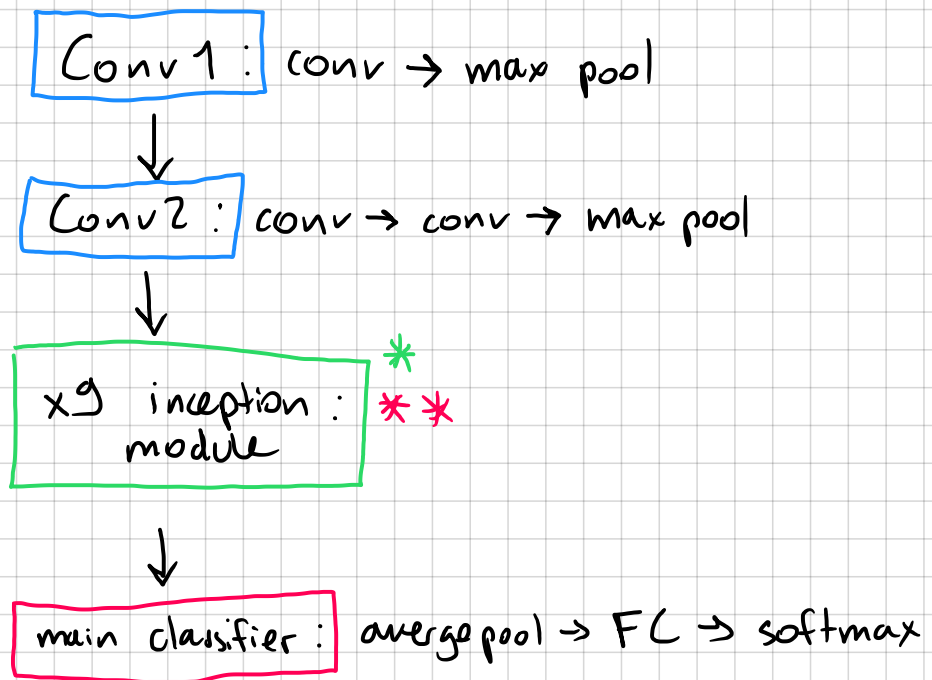


GoogLeNet (2014):

22 layers

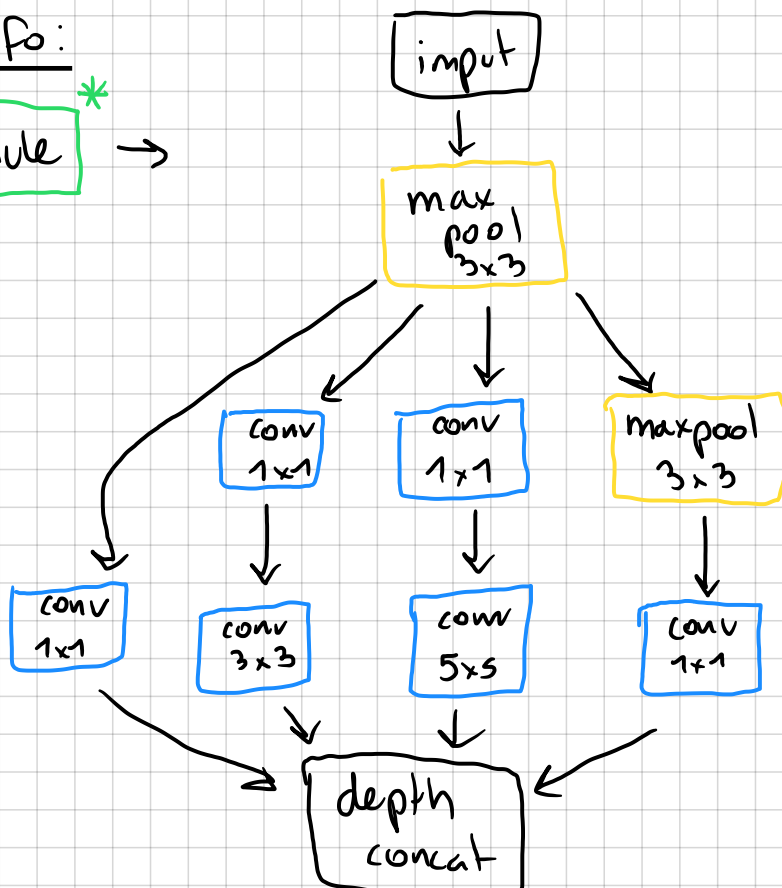
mix of "normal" convolution and inception block/layers

full architecture:



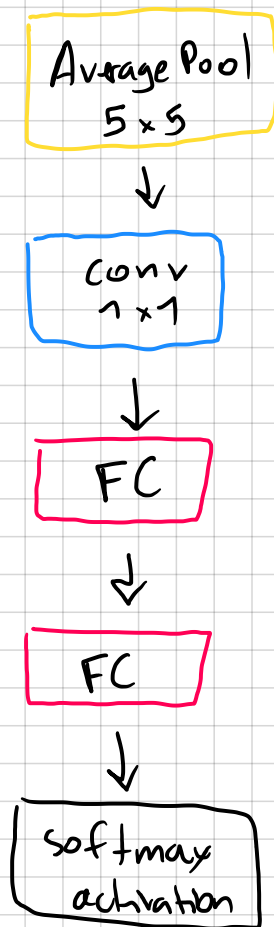
Important info:

Inception module *



auxiliary classifiers:

at the 4th and 7th inception modules we add auxiliary classifiers that look like this:



same as the main classifier but 1 less FC and 1 conv layer

These layers are used for regularization used to get a better gradient flow in the early layers

Key points for GoogleNet:

- inception modules → multiple feature extractors, helps to learn
- 1x1 convolutions → reduces dimensions, fewer params to learn
- very deep network but very computationally efficient!
- auxiliary classif → better gradient flow

VGG Net (2015)

16 to 19 layers

idea $\rightarrow$ investigate effect of depth $\rightarrow$ keep going deeper

always 3×3 convolution, but keep adding more filters

input: $224 \times 224 \times 3$

add blocks: $(3 \times 3 \text{ conv} + \text{ReLU}) \times N \rightarrow \text{Maxpooling } (2 \times 2)$

$\downarrow$ halves dimensions

start with 64 filters

1) $\text{conv } 3 \times 3^{64} \rightarrow \text{conv } 3 \times 3^{64} \rightarrow \text{maxpool}$
output: $112 \times 112 \times 64$

2) $\text{conv } 3 \times 3 \times 128 \rightarrow \text{conv } 3 \times 3 \times 128 \rightarrow \text{maxpool}$
output: $56 \times 56 \times 128$

$\therefore$ blocks 3, 4 & 5 use 3 conv 3×3

3) $\rightarrow 256$

4) $\rightarrow 512$

5) $\rightarrow 512 \times 7 \times 7$ $\leftarrow$ input of first FC

Flatten output $\rightarrow$ 4096 $\rightarrow$ 4096 $\rightarrow$ 1000 $\rightarrow$ softmax output
input 25k $\uparrow$ output size

Conclusion: lots of params, no batch norm, no residual connections or inception module

$\rightarrow$ difficult to train, but works great as a feature extractor

ResNet $\rightarrow$ beats it

ResNet (2015):

152 layers, 60M parameters

Residual Networks $\rightarrow$ very deep network but trainable thanks to residual connections (skip connections)

fixes vanishing gradients

instead of learning $H(x)$ we learn $F(x) = H(x) - x$
 $\Leftrightarrow y = F(x) + x$

example:

input $x \rightarrow \text{conv } 3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{conv } 3 \times 3 + x \rightarrow \text{ReLU}$

used to keep some signal for gradients

if input is optimal, then this layer's weights can be set to 0
easy to make small changes without losing the signal

bottleneck residual block:

a basic block takes input, passes in 3×3 convs, expensive!

example:

input 256, we would do $3 \times 3 \times 256 \times 256 \approx 590 \text{ k params}$

with bottleneck, we reduce the input first, then do the operation, then expand back:

input 256,

bottleneck width! $\rightarrow$ compression

$(1 \times 1 \times 256 \times 64) + (3 \times 3 \times 64 \times 64) + (1 \times 1 \times 64 \times 256) \approx 70 \text{ k}$

$\times 8$ less params! then we add the residual x

ResNet also adds batch normalization

Currently the best!

4. Vision Transformers

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

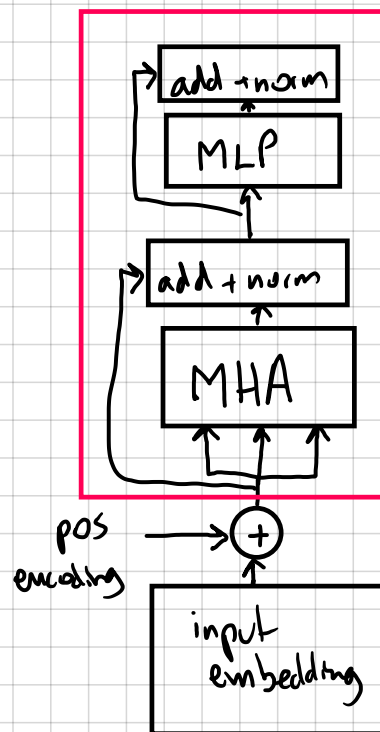
For images:

- 1) split image into patches
- 2) flatten patch
- 3) linear projection of each patch (embedding)
- 4) add positional embedding
- 5) add class token (optional)

Transformer Encoder:

multiple blocks of:

- Multi-head self attention
- MLP
- residual connection
- layer Norm



encoder block

These blocks encode the relationship between patches in an image also global context

can be used for: classification, feature extraction, detection/segmentation

Decoder Block:

Used in image to image or image to text task

takes as input query tokens

almost same as encoder but add a cross-attention block where input is output of masked self-attention of query

plus other token query or image features (from encoder block for example)

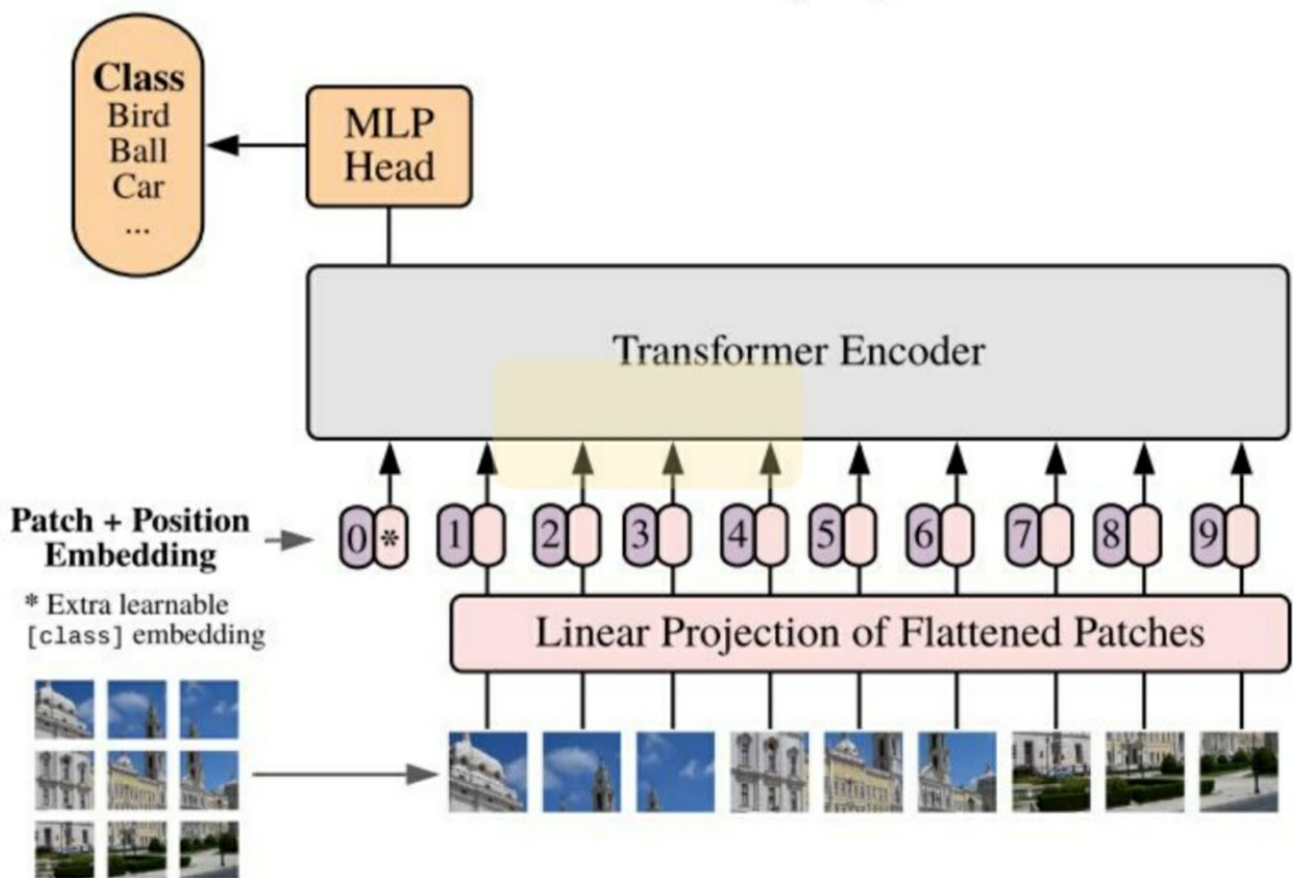
the decoder outputs text, object queries or segmentation masks for example

Convolution Networks $\rightarrow$ local attention

convolution operations try and combine local info over deep networks to try and get more global info

ViT $\rightarrow$ gets global attention?

Encoder architecture:



5. Transfer learning & Domain adaptation

Transfer Learning:

use knowledge gain in one task on another task

example:

train a network on ImageNet, like AlexNet
this is our source task

now we have a target task, new dataset, like Cal-101 with some new classes (maybe 150)

idea: take pre-trained model, freeze model up to layer i ,
for example last conv layer
remove the FC classifier layers

add new layers for classification on top of the AlexNet backbone
train new classifier (like an SVM) on the new data.
or fine tune if architecture is good

Much faster than training full new model

You can also fine tune more layers, like the last conv layers

other option, don't freeze deep layers, but use much smaller learning rate

Domain Adaptation:

sometimes, task is the same but domain is different
ex: numbers hand-written vs. typed, or RGB vs. B&W

in the case of **unsupervised domain adaptation**,
source data is labeled, target data isn't

source vs. target $\rightarrow$ **domain shift**

Domain adaptation aims to reduce the domain shift in the features extracted

UDA strategy: domain-adversarial training

we have a feature extractor G and a domain classifier D

goal:

- G will align the domain of source and target features
- D will identify the different domains

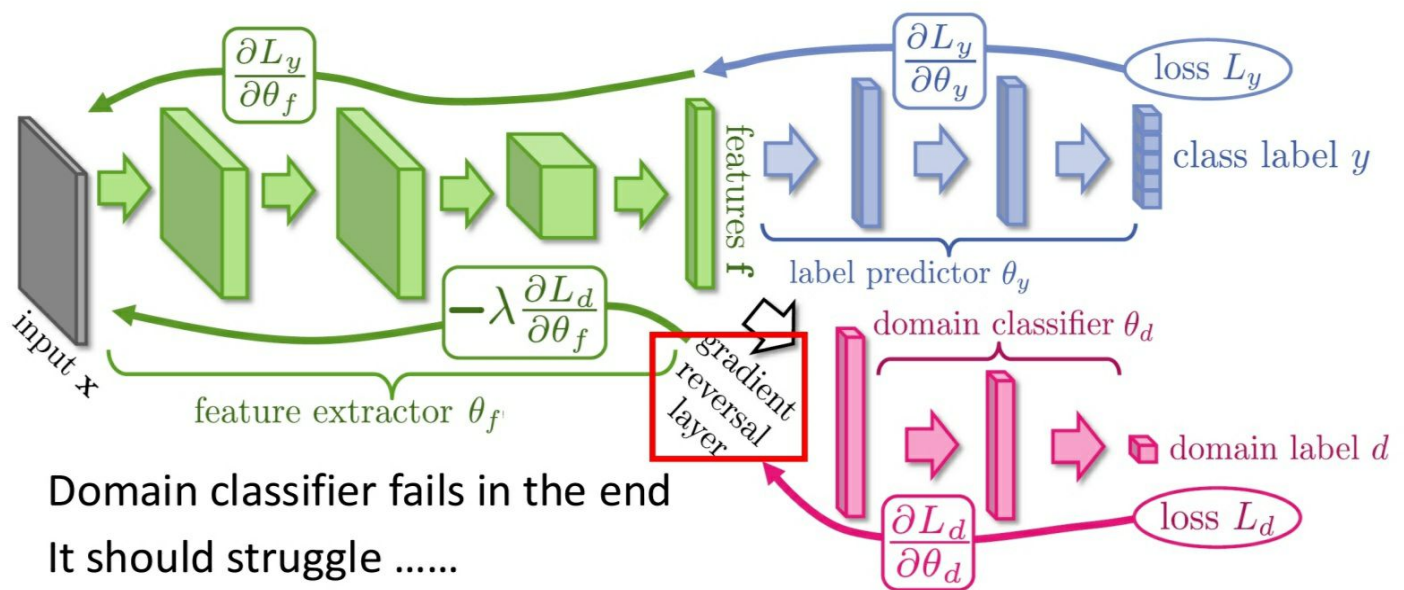
add our label predictor $P \rightarrow$ predict label y from input

for the training to be successful:

- $P \rightarrow$ correct prediction for source & data points
- $G \rightarrow$ generate features that are similar for points of same label but different domains
- $D \rightarrow$ cannot differentiate which features are from what domain

Introducing the **gradient reversal layer**!

GRL:



Domain classifier fails in the end
It should struggle

Optim from [Yaroslav Ganin, Victor Lempitsky, ICML, 2015], reconsidered and better formulated in GAN framework (latter in the course)

in order to learn how to "fool" D, we backpropagate the opposite gradient (times a factor)

that way G learns to generate features that maximize the loss of the discriminator

we call these **DANN** → domain adversarial neural networks

So overall:

- Transfer learning → same domain, different task
- domain adaptation → same task, different domain

6. VLMs : Representation Learning

representation learning: learn useful features (embeddings) from data

good representations are invariant to rotations, brightness, color etc...

The main idea is that **both text and images** get encoded in the **same embedding space**

CLIP: Contrastive Language-Image Pretraining

CLIP is a **dual architecture** model

for text: text encoder $\rightarrow$ transformer

for images: ResNet ViT

Learning strategy:

Training set of pairs of images and their labels

original training used 400M pairs

Loss: uses **contrastive loss**

Contrastive / Triplet Loss: variant of standard margin loss of SVM

$$\text{TripletLoss}(I, T, T') = \max\{0, \alpha + d(I, T) - d(I, T')\}$$

α : margin param

I : anchor, image representation

T : positive, real image caption

T' : negative, contrastive image caption

CLIP can also use the Negative Contrastive Estimation Loss

NCE Loss:

closer to a true classification loss

tries to distinguish true pairs from wrong pairs

$$\mathcal{L}_{\text{infoNCE}_{\text{CLIP}}} = - \sum_i \log \left(\frac{\exp\left(\frac{\text{sim}(I_i, T_i)}{\tau}\right)}{\sum_{k=1}^N \exp\left(\frac{\text{sim}(I_i, T_k)}{\tau}\right)} \right)$$

explanation:

for each image I_i , compute the cosine similarity between I_i and the true "positive" label T_i . Divide by the temperature τ and take exponential. (1)

Now do the same process but for I_i and all the other labels, and sum result (2)

Divide (1) by (2), take log

intuition:

Pull true pairs together, push incorrect pairs away from each other

More stable than triplet loss, because it uses many more negative examples to train.

NCE >> triplet/contrastive loss

7. VLMs: using LLMs

idea: use Vision encoder with an LLM decoder
→ give image and output caption / description

encoder outputs a feature mapping, and decoder takes that as input.

Feature injection: decoder takes input tokens, which represent the image

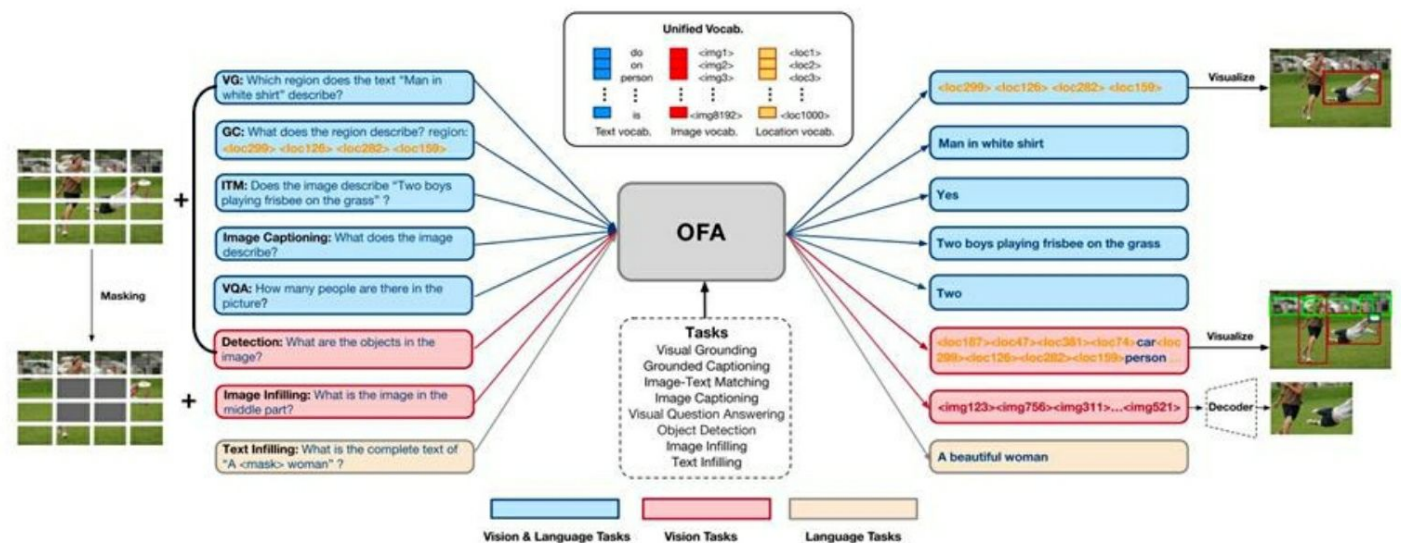
but no task?

inject tokens!

embed instruction like "Caption this image" and add to the encoder output for the decoder

1M4all: One Model for All

**One model with:
many inputs / many outputs / many tasks**



Needs a lot of good data to train!

8. Segmentation

Semantic segmentation: label each pixel with a category label, like horse or person

object detection: detect every instance of an object and localize it with a bounding box.

Instance segmentation: detection + segmentation
we want to segment and detect different instances, like person 1 or 2

Supervised Segmentation:

Famous datasets:

- PASCAL VOC 12: 21 classes, ~1500 images for train/test/val
- COCO: 91 classes + 71 super categories
80k images in train, 20k in val
used for captioning, instance segmentation and detection

Full vs. Weakly supervision:

- full: actually annotate everything, pixel level!
expensive to have, but efficient
- weakly: less precise labels, like general tags, points or bounding boxes
much easier to label!

Fully Convolution Networks (FCN):

ImageNet: 1.2 million centered images
easy to use ConvNet architectures, like VGG

but what about complex images?

like bigger scale, multiple objects etc...

Naive approach: resize image $\rightarrow$ doesn't really work...

Sliding Window approach:

we gradually reduce the feature space

for VGG for example:

output is 4096 convolutions, $7 \times 7 \times 512$ filters

the idea is to output n feature maps, with n number of classes

each feature map can be interpreted as a confidence map with
is class specific (per channel)

we go from $h \times w \times k$ to $h \times w \times C$ to $1 \times 1 \times C$

$\uparrow$
 k features
like 4096 above

$\uparrow$
 C classes
score per location

the final block $1 \times 1 \times 1$ gives us a confidence score over all classes
for ImageNet, take argmax because only 1 image

but could take with threshold $\rightarrow$ replace softmax with sigmoid?

Segmentation:

FCN approach

the FCN we defined above pretty much classify each pixel

problem:

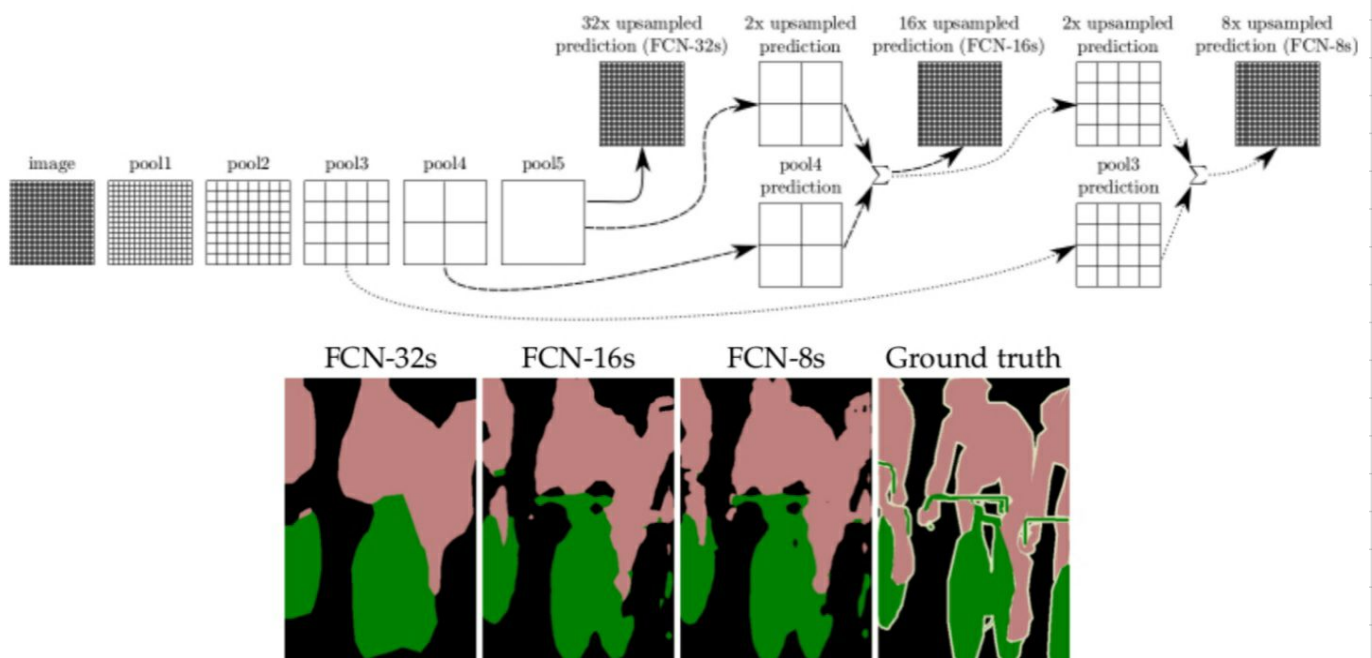
the FCN pipeline reduces dimensions!
goes from 224×224 to 7×7 for example
loss of information

solution $\rightarrow$

try to upsample from each feature map

Solution of the FCN approach

- Problem: max pooling and striding reduces spatial resolution
- Dense prediction: combines feature hierarchies
- Initialized with the parameters of coarse net
- Fine-tuning all layers by backpropagation



this is the basic FCN approach

learn to reconstruct original dimensions from the obtained feature maps

Deconvolution Networks:

New type of network with 2 main parts:

- convolution: feature extractor (classic conv network)
- deconvolution: a shape generator, that produce object segmentation with given features

convolution part:

takes image $h \times w$ input

produces 1×1 features after convolution and pooling layers

deconvolution part:

takes input 1×1 features (output of conv)

applies many deconvolution and unpooling layers

we want the second part to mirror the first one

ex: if we do

$$224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7 \rightarrow 1 \times 1$$

then the deconvolution does:

$$1 \times 1 \rightarrow 7 \times 7 \rightarrow \dots \rightarrow 112 \times 112 \rightarrow 224 \times 224$$

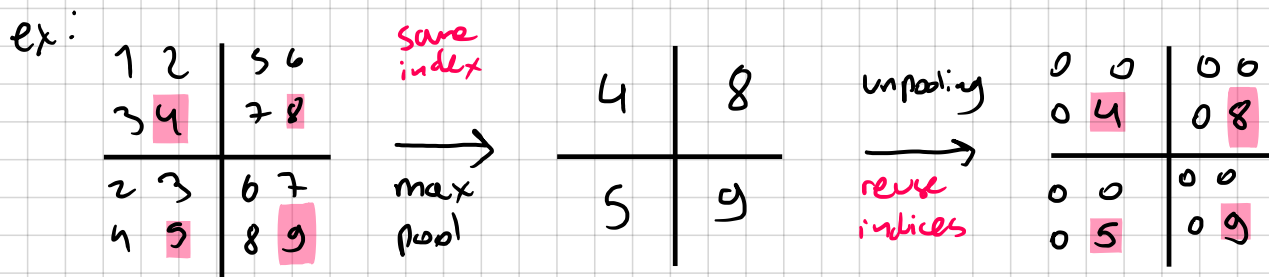
Unpooling:

usually used as the inverse of max pooling

during max pooling in the conv network, save index of max values

→ in unpooling put the value in the original position

unpooling is not lossless! Not pure recreation



we can see we lost some info but close enough

Deconvolution:

map one activation to many activations

filters learn to recreate this mapping (kind one-to-many)

so this step "refines" the given feature map

Shortcut connections:

in conv (encoder in a sense), higher layers get shallow info, like corners, edges etc...

deeper layers extract more semantic information (dog or cat?)

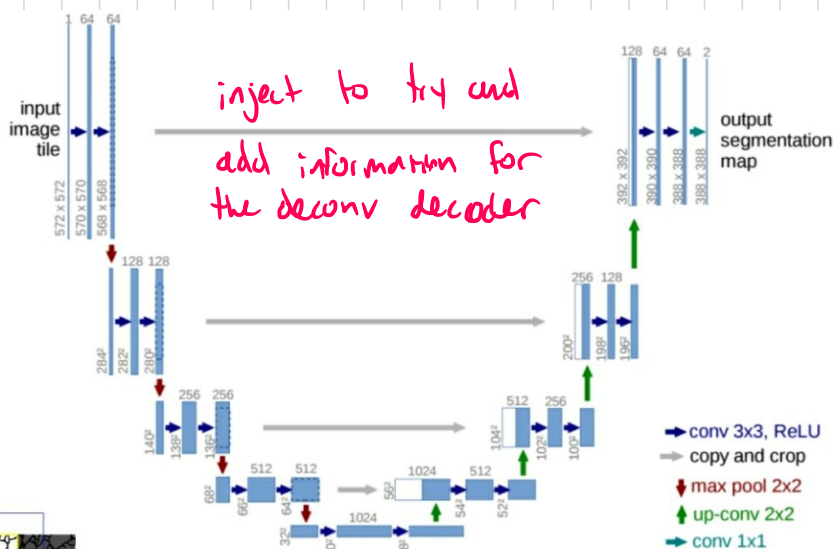
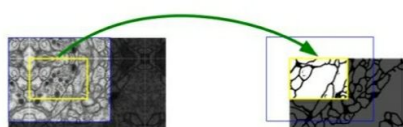
but by reducing dimensions you can kill information and lose it

idea: directly inject encoder features into decoder

but has to be same dimensions! we call it feature copying!

U-Net:
Hourglass/U shape Net
+ shortcut connection by feature copies

Very popular in medical
Works well with low
training data



A-trous approach:

9. Self Supervised Learning (SSL)